Next, let's see how we can use the `_analyze` API to check the output before we index the text (again, we don't specify the analyzer because Elasticsearch uses the `standard` analyzer if we don't mention one explicitly):

```
GET _analyze
{
  "text": "Hot cup of ☕ and a 🍿 is a Weird Combo :(!!"
}
```

The output of this `GET` command is as follows (other than the first token, the rest are condensed for brevity):

```
{
  "tokens" : [
    {
      "token" : "hot",
      "start_offset" : 0,
      "end_offset" : 3,
      "type" : "<ALPHANUM>",
      "position" : 0
    },
    { "token" : "cup", ... },
    { "token" : "of", ... },
    { "token" : "☕", ... },
    { "token" : "and", ... },
    { "token" : "a", ... },
    { "token" : """🍿""", ... },
    { "token" : "is", ... },
    { "token" : "a", ... },
    { "token" : "weird", ... },
    { "token" : "combo", ... }
  ]
}
```

The lowercase token filter lowercases the words.

Stop words are not removed because the stop filter is disabled by default.

The coffee cup is indexed as is—the width is one character.

The popcorn emoji is indexed as is—the width is two characters.

The smiley and exclamation marks are removed by the standard tokenizer.

The words are split based on whitespace and non-letters (punctuation), which is the hallmark of the `standard` tokenizer. The tokens are then passed through the `lowercase` token filter. Figure 7.5 shows a condensed output of this command in Dev Tools.

Components of a built-in analyzer

As discussed, each of the built-in analyzers comes with a predefined set of components such as character filters, tokenizers, and token filters—for example, a `fingerprint` analyzer is composed of a `standard` tokenizer and several token filters (`fingerprint`, `lowercase`, `asciifolding`, and `stop`) but no character filters. It isn't easy to tell the anatomy of an analyzer unless you memorize them over time! So, my advice is to check the definition on the documentation page if you need to know the nitty-gritty details of an analyzer.